

ALGORITHME optimiser_pour_low_resource(donnees_limitees)

ENTREE: corpus limité, dictionnaire limité

SORTIE: pipeline NLP optimisé

// Stratégie 1: Augmentation de données intelligente

corpus_augmente = augmenter_donnees(corpus_original)

// Stratégie 2: Extraction maximale d'information

informations = extraire_toutes_informations(corpus_augmente)

// Stratégie 3: Modèles légers hiérarchiques

SI taille(donnees) < SEUIL_PETIT ALORS

 utiliser_modeles_simples()

// N-grams, règles, distances

SINON SI taille(donnees) < SEUIL_MOYEN ALORS

 utiliser_approche_hybride()

// Combinaison simple/statistique

SINON

 utiliser_modeles_legers()

// TF-IDF, Bayes naïf

FIN SI

// Stratégie 4: Validation croisée stricte

performance = evaluer_modele(cross_validation_stricte)

// Stratégie 5: Fallback progressif

ALGORITHME fallback_progressif(requete)

ESSAYER:

// 1. Règles exactes

resultat = appliquer_regles_exactes(requete)

SI resultat.confidence > 0.9 ALORS

RETOURNER resultat

FIN SI

// 2. Similarité de chaînes

resultat = chercher_similarites(requete)

SI resultat.confidence > 0.7 ALORS

RETOURNER resultat

FIN SI

// 3. Patterns généraux

resultat = appliquer_patterns_generaux(requete)

SI resultat.confidence > 0.5 ALORS

RETOURNER resultat

FIN SI

// 4. Fallback: retourner "je ne sais pas"

RETOURNER {"confidence": 0, "message": "Inconnu"}

FIN ESSAYER

FIN ALGORITHME

RETOURNER pipeline_optimise

FIN ALGORITHME